Script-Controlled ACL – Restrict Record Access Based on Field Value

Project Design Phase: Solution Architecture

Date:	October 3, 2026	**Maximum Marks:**	4 Marks
Team ID:	SWTID-2026-8089	**Project Name:**	Script-Controlled ACL

Solution Architecture:

Solution architecture is a complex process – with many sub-processes – that bridges the gap between business problems and technology solutions. Its goals are to:

- Find the best tech solution to solve existing business problems.
- Describe the structure, characteristics, behavior, and other aspects of the software to project stakeholders.
- Define features, development phases, and solution requirements.
- Provide specifications according to which the solution is defined, managed, and delivered.

Solution Architecture Diagram:

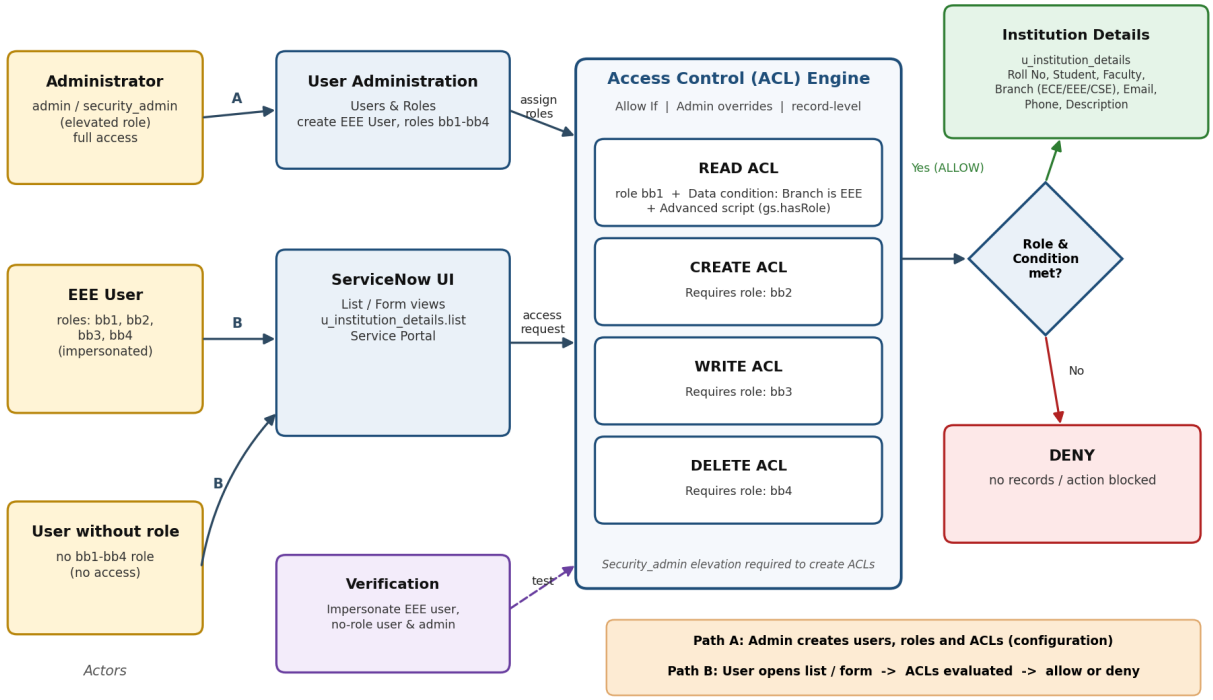


Figure 1: Architecture and data flow of the script-controlled ACL application

Architecture components:

The solution architecture for Script-Controlled ACL on ServiceNow connects user touchpoints, the access control layer and the data layer into a secure record-level access workflow, where every request is evaluated against user roles and record field values.

Architecture Components & Flow

User Interaction Layer (Frontend / Channels):

- **ServiceNow UI (List / Form / Service Portal):** Users open the Institution Details list (u_institution_details.list) or forms to view, create, edit and delete records.
- **Administrator:** Holds the admin role with full access and elevates to security_admin to create and modify ACLs.
- **EEE User (bb1, bb2, bb3, bb4):** Test user, assigned the custom roles, accessing records only as permitted by the ACLs.
- **User without role:** A user with none of the custom roles, who sees no records.

Core Security & Access Control Layer (ServiceNow Platform):

- **User Administration (Users & Roles):** Stores the EEE User record and the custom roles bb1 (read), bb2 (create), bb3 (write) and bb4 (delete), which are assigned to the EEE User.
- **Access Control (ACL) Engine:** Evaluates every request on u_institution_details with the 'Allow If' decision type and admin override.
- **READ ACL (bb1):** Requires role bb1, a data condition (Branch is EEE) and an advanced script using gs.hasRole(), so only EEE branch records are visible to bb1 users while admins retain full access.
- **CREATE ACL (bb2):** Requires role bb2; enables the New button.
- **WRITE ACL (bb3):** Requires role bb3; allows editing of EEE branch records.
- **DELETE ACL (bb4):** Requires role bb4; allows deletion of EEE branch records.
- **Decision Point:** If the role and data condition are satisfied the request is allowed; otherwise it is denied and no records or actions are shown.

Data Layer (Backend / Storage):

- **Institution Details table (u_institution_details):** Custom table with Student Roll Number (auto number), Student Name and Faculty Name (reference to User), Branch (choice: ECE, EEE, CSE), Email, Phone Number and Description.
- **Sample records:** Multiple records with different Branch values (ECE, EEE, CSE) are used to demonstrate record-level filtering.

Verification Layer:

- **Impersonation:** Impersonating the EEE user shows only EEE branch records (with New, edit and delete options based on roles); a user without roles sees no records; the admin sees all records regardless of branch.